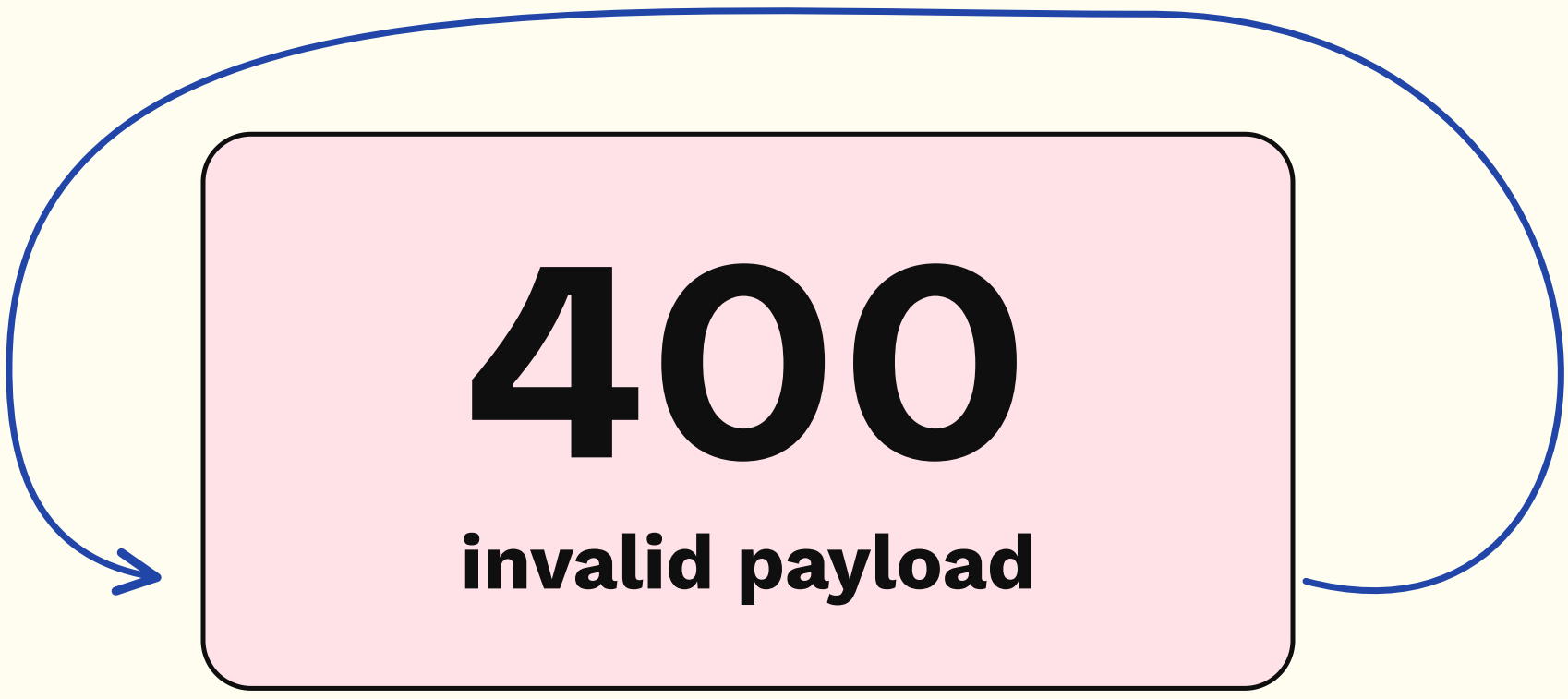
Retrying a bad request won't fix it.

Diagnose before your AI tool retries.



same payload, same failure

Fix. Wait. Verify. Choose before retrying.

Madhan Kumar

Three failures. Three decisions.

The response alone is only one clue.

Invalid payload

Validation cannot pass unchanged → **FIX**

Temporary overload

Capacity may recover → **WAIT**

Unknown write result

The operation may have committed → **VERIFY**

Two checks: failure type + replay safety.

Use the provider's documented error semantics.

A malformed request needs a change.

Waiting does not repair an invalid payload.

```
{"limit": -1}
```

API contract: limit must be positive.



Repair the input or stop.

Illustrative API contract; provider errors can differ.

Rate limited? Make room to recover.

Retry only a documented temporary condition.



give recovery some room

Honor Retry-After when supplied.

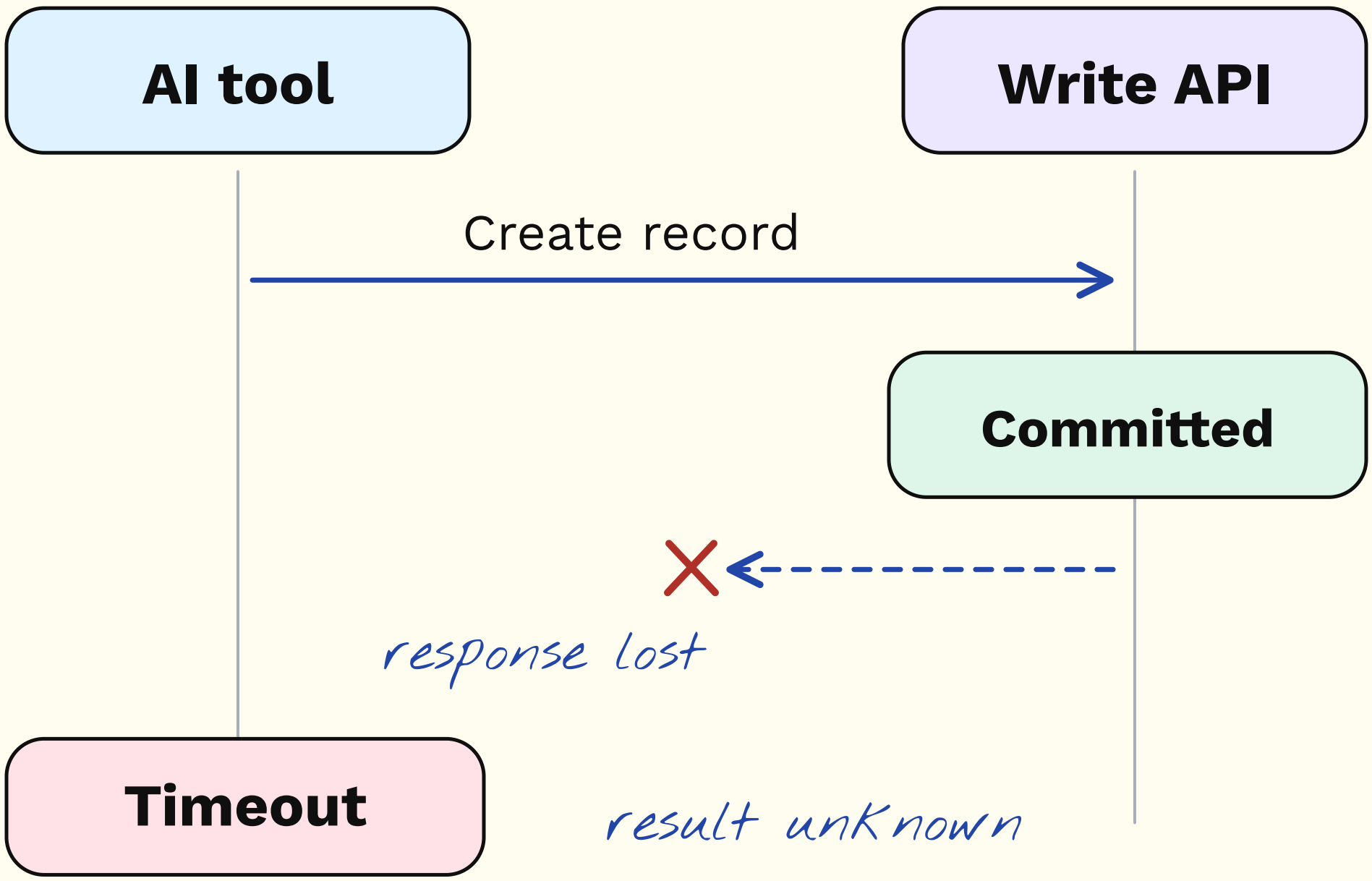
Add jitter to spread retry timing.

Cap attempts and total elapsed time.

A quota or billing limit may need a different fix.

A timeout can hide a completed write.

The response can disappear after the commit.



Check outcome or use enforced idempotency.

One possible failure path. Idempotency requires API support.

Change the policy. Count the attempts.

Offline simulation · cap: 3 total attempts

SCENARIO / POLICY	CALLS	RESULT
Invalid payload Retry every failure	3	FAIL
Invalid payload Classify before retrying	1	STOP
Temporary rate limit Wait, then retry safely	2	OK
Ambiguous write Verify before repeating	1	CHECK

A stopped retry can be the correct result.

Synthetic responses. No network or latency benchmark.

Before the next retry, ask three things.

Failure type + replay safety + remaining budget.

1 **Can this failure clear without a request change?**

2 **Can this operation be repeated safely?**

3 **Does a retry fit the remaining time budget?**

If no: stop or change the action.

Sources: RFC 9110 · RFC 6585 · AWS Builders' Library

Madhan Kumar